Programa de Ingeniería de Sistemas
Laboratorio de Ingeniería de Software II
Periodo 2-2026

Taller 8: Patrones de Diseño, de Comportamiento y Estructurales
(Será realizado en la Sala de Computo)

Objetivos

● Aplicar los patrones de diseño de comportamiento *State* y *Template Method*, y los patrones de diseño estructurales *Decorator, Adapter y Facade*, mediante el desarrollo de funcionalidades para el proyecto Banco de Preguntas Saber PRO.

El estudiante deberá identificar situaciones en las cuales estos patrones permitan construir una solución más flexible, extensible y mantenible, evitando que la incorporación de nuevos tipos de preguntas implique modificar excesivamente el código existente.

El Escenario del problema

El Banco de Preguntas Saber PRO es una aplicación destinada a gestionar preguntas que pueden ser utilizadas en la construcción de instrumentos de evaluación.

Inicialmente, el sistema trabaja con preguntas de:

a.- Selección múltiple con única respuesta.

b.- Selección múltiple con múltiples respuestas.

Sin embargo, la institución desea que el sistema pueda incorporar progresivamente nuevos tipos de preguntas sin tener que modificar de manera significativa las funcionalidades existentes.

Para este taller se propone incorporar adicionalmente:

c.- Verdadero/Falso.

En consecuencia, el sistema deberá permitir gestionar diferentes tipos de preguntas conservando una estructura común y permitiendo que cada tipo implemente su comportamiento particular.

Además, las preguntas atraviesan diferentes estados durante su ciclo de vida (ver Figura 1):

Una pregunta no puede ser utilizada en una prueba mientras no haya sido aprobada y publicada.

El sistema también debe permitir:

- Crear preguntas.
- Registrar opciones de respuesta.
- Validar preguntas.
- Cambiar el estado de una pregunta.
- Evaluar respuestas.
- Agregar características adicionales a una pregunta.
- Integrar preguntas provenientes de fuentes externas.

- Simplificar la interacción de los clientes con los diferentes subsistemas.

¿Qué se debe hacer?

Cada equipo de trabajo deberá aplicar los patrones de diseño en diferentes partes del Banco de Preguntas Saber PRO. Basarse en los ejemplos que se explicaron en la para la implementación de los patrones que se deben aplicar en este taller.

Los patrones que se deben aplicar según los requerimientos del proyecto Banco de Preguntas Saber PRO son:

No.	Tipo de patrón	Nombre del patrón	Aplicación en el proyecto
1	Comportamiento	State	Ciclo de vida de una pregunta
2	Comportamiento	Template Method	Proceso de evaluación de respuestas
3	Estructural	Decorator	Agregar características a las preguntas
4	Estructural	Adapter	Integrar preguntas provenientes de sistemas externos
5	Estructural	Facade	Simplificar la interacción con el banco de preguntas

Tabla 1: Descripción de patrones a usar en el taller

Patrón State: ciclo de vida de una pregunta

Una pregunta del banco no permanece siempre en el mismo estado. Se propone el ciclo de vida de una pregunta que se muestra en la Figura 1.

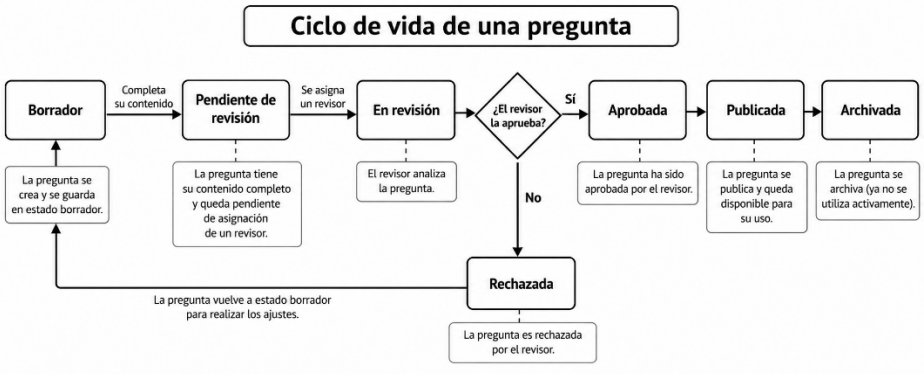


Figura 1: Ciclo de vida de una pregunta

Se debe aplicar State para evitar que la clase Question tenga grandes cantidades de instrucciones if/else para controlar su comportamiento. Con base en los estados sugeridos de la Figura 1, se sugieren los siguientes estados:

DraftState
ReviewState
PendingReviewState
ApprovedState

PublishedState
ArchivedState

La interface QuestionState podría ser:

```
public interface QuestionState {

    void submitForReview (Question question);

    void approve (Question question);

    void publish (Question question);

    void archive (Question question);

    String getName ();
}
```

La clase Question deberá mantener internamente su estado:

```
public class Question {

    private QuestionState state;

    public Question () {
        state = new DraftState ();
    }

    public void setState(QuestionState state) {
        this.state = state;
    }

    public QuestionState getState () {
        return state;
    }
}
```

Implementar el patrón State de manera que las operaciones permitidas dependan del estado actual de la pregunta.
Por ejemplo:

No.	Operación	Permiso
1	Borrador → En revisión	Permitido
2	En revisión → Aprobada	Permitido
3	Aprobada → Publicada	Permitido
4	Borrador → Publicada	No Permitido
5	Publicada → Borrador	No Permitido

Tabla 2: Permisos para operaciones

Patrón Template Method: evaluación de preguntas

Los diferentes tipos de preguntas tienen un proceso general de evaluación.

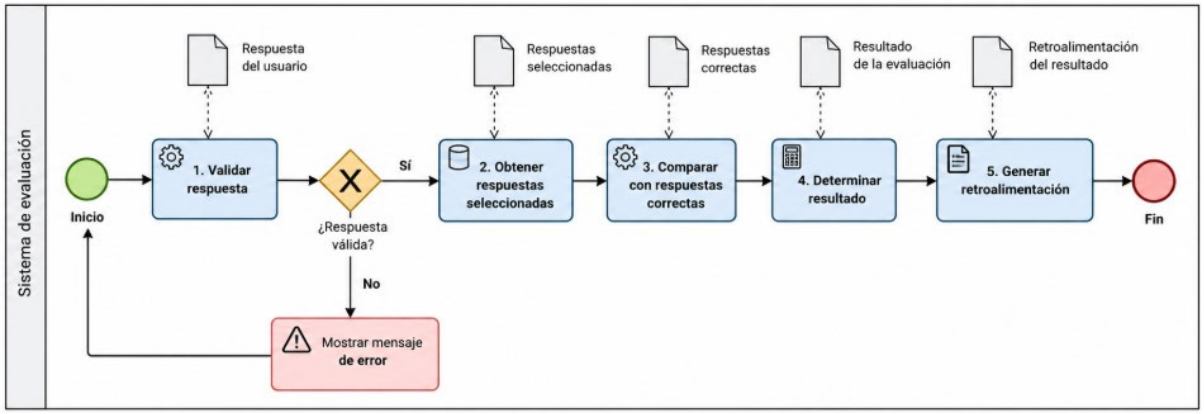


Figura 1: Ciclo de vida de una pregunta

Sin embargo, la forma de determinar si una respuesta es correcta depende del tipo de pregunta. Por ejemplo:

Selección única

El estudiante debe seleccionar exactamente una respuesta correcta.

Selección múltiple

El estudiante puede seleccionar varias respuestas correctas.

Verdadero/Falso

El estudiante selecciona Verdadero o Falso.

Por lo tanto, se propone:

```
public abstract class QuestionEvaluator {

    public final boolean evaluate(Question question,
                                  List<String> answers) {
        validate (question, answers);
        return compareAnswers (question, answers);
    }

    protected abstract void validate (
            Question question,
            List<String> answers);

    protected abstract boolean compareAnswers (
            Question question,
            List<String> answers);
}
```

Aplicar *Template Method* para evitar duplicar el algoritmo general de evaluación. El algoritmo general debe permanecer en la clase abstracta y las clases concretas deberán implementar únicamente los pasos que cambian.

Patrón Template Method: evaluación de preguntas

Una pregunta puede tener características adicionales sin modificar directamente la clase Question.
Por ejemplo:
- Pregunta de alta dificultad.
- Pregunta contextualizada.
- Pregunta con retroalimentación.
- Pregunta asociada a una competencia.
- Pregunta marcada para revisión.

Se propone utilizar *Decorator* para agregar estas características dinámicamente.
Por ejemplo:

```
public interface QuestionComponent {

    String getDescription ();

    double getScore ();
}
```

Una implementación básica podría ser:

```
public class BasicQuestion implements QuestionComponent {

    private String description;

    public BasicQuestion (String description) {
        this.description = description;
    }

    @Override
    public String getDescription() {
        return description;
    }

    @Override
    public double getScore() {
        return 1.0;
    }
}
```

Decorador abstracto:

```
public abstract class QuestionDecorator
        implements QuestionComponent {

    protected QuestionComponent question;

    public QuestionDecorator (QuestionComponent question) {
        this.question = question;
    }

    @Override
    public String getDescription () {
        return question.getDescription();
    }

    @Override
    public double getScore () {
        return question.getScore();
    }
}
```

Y un decorador concreto:

```
public class HighDifficultyQuestion
        extends QuestionDecorator {

    public HighDifficultyQuestion (
            QuestionComponent question) {

        super(question);
    }

    @Override
    public String getDescription () {

        return question.getDescription()
                + " [ALTA DIFICULTAD]";
    }
}
```

De esta manera:

```
QuestionComponent question = new BasicQuestion ("¿Qué es arquitectura de software?");

question = new HighDifficultyQuestion(question);
```

El objeto adquiere una característica adicional sin modificar BasicQuestion.

Patrón Adapter: importar preguntas externas

Supongamos que una institución externa proporciona preguntas mediante un servicio que devuelve información en un formato diferente al utilizado por el Banco de Preguntas.
Por ejemplo:

```
public class ExternalQuestionService {

    public String getQuestion() {

        return """
        {
            "statement": "¿Cuál es la capital de Colombia?",
            "answer": "Bogotá"
        }
        """;
    }
}
```

Pero nuestro sistema espera:

```
public interface QuestionProvider {

    Question getQuestion ();
}
```

El servicio externo no puede utilizarse directamente.
Por lo tanto, se debe crear:

```java
public class ExternalQuestionAdapter
        implements QuestionProvider {

    private ExternalQuestionService service;

    public ExternalQuestionAdapter (
            ExternalQuestionService service) {

        this.service = service;
    }

    @Override
    public Question getQuestion() {

        String data = service.getQuestion();

        // Transformar los datos externos
        // al modelo interno Question

        return ...;
    }
}
```

Implementar el Adapter que permita transformar una pregunta externa al modelo utilizado por el Banco de Preguntas.

Patrón Facade: para el Banco de Preguntas

El Banco de Preguntas tiene diferentes subsistemas:
QuestionService
QuestionRepository
QuestionValidator
QuestionEvaluator
QuestionPublisher
QuestionImporter

Un cliente no debería tener que conocer todos estos componentes.
Por ello se deberá implementar:

```java
public class QuestionBankFacade {

    private QuestionService questionService;
    private QuestionValidator validator;
    private QuestionPublisher publisher;

    public void createQuestion (Question question) {

        validator.validate(question);

        questionService.save(question);
    }
    public void publishQuestion (Question question) {

        validator.validate(question);

        questionService.approve(question);

        publisher.publish(question);
    }
}
```

La aplicación cliente podrá utilizar:

```java
QuestionBankFacade facade = new QuestionBankFacade(...);
facade.createQuestion(question);
```

en lugar de interactuar directamente con todos los subsistemas.

Requerimientos funcionales mínimos

La aplicación debe permitir:

1. Crear una pregunta.
2. Seleccionar el tipo de pregunta.
3. Agregar opciones de respuesta cuando corresponda.
4. Definir respuestas correctas.
5. Consultar preguntas.
6. Cambiar el estado de una pregunta.
7. Evaluar una respuesta.

8. Agregar características mediante Decorator.
9. Importar al menos una pregunta mediante Adapter.
10. Utilizar la Facade para las operaciones principales.

Estructura del proyecto

Crear la estructura del directorio de trabajo similar a la mostrada en la Figura 2:

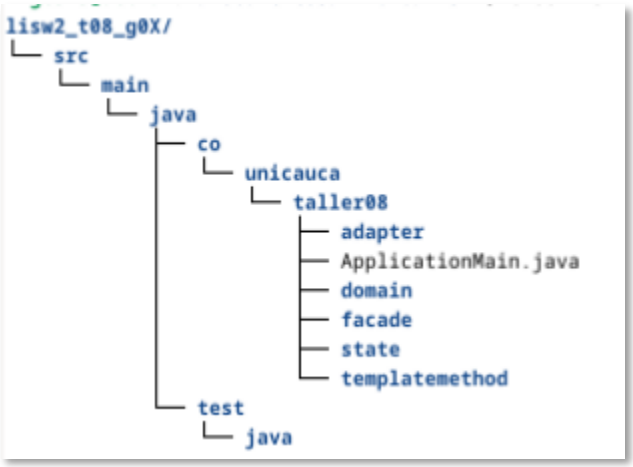


Figura 2: Estructura del proyecto

Rúbrica de evaluación

Criterio	Excelente (5 pts)	Bueno (4 pts)	Aceptable (3 pts)	Insuficiente (0, 1 pts)	Peso (%)
State	Implementa correctamente el ciclo de vida y las transiciones	Presenta pequeñas inconsistencias	Funciona parcialmente	No implementa State	25%
Template Method	Implementa correctamente los algoritmos de evaluación	Presenta pequeñas omisiones	Implementación parcial	No implementa el patrón	25%
Decorator	Incorpora características	Presenta pequeñas inconsistencias	Implementación limitada	No representa Decorator	25%

	dinámicamente y correctamente				
Facade	Simplifica correctamente la interacción con los subsistemas	Presenta pequeñas inconsistencias	Implementación parcial	No implementa Facade	25%

¿Qué se debe entregar?

Se puede trabajar en grupos de dos personas.

1.- Se debe implementar la aplicación en un IDE.

2.- En el informe se debe incluir un breve resumen del proyecto de clase y un enlace al repositorio GitHub del grupo de trabajo.

Subir el informe a la tarea dispuesta por el docente en univirtual.

Fecha de entrega: martes, 13 de octubre de 2026 hasta las 18:00

Referencias

Material de apoyo:
Wilson L Pantoja, Julio A Hurtado.